

コードの時系列変化を考慮した 保守性低下の要因分析と改善

鈴木研究室 2410064

笹川 尋翔

目次

1. 背景
2. 目的
3. 関連研究
4. 分析手順
5. 結果
6. 考察
7. 課題と展望

1. 背景

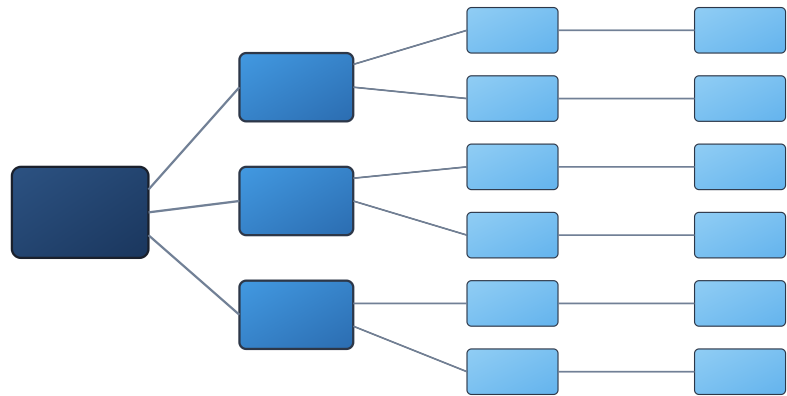


図1 ソフトウェア構造の複雑化

76%の開発者がリファクタリングによるバグ混入のリスクを認識 [1]

ソフトウェア構造が常に変化し、一時的な分析では品質改善が困難に

[1] Microsoft Research, "An Empirical Study of Refactoring Challenges and Benefits at Microsoft", 2014

2. 目的

- 保守性指標の時系列変化量を品質改善に役立てる
 - これまで考慮されていなかった保守性の変化に着目
 - 例: メソッドの複雑度の変化、テストやドキュメント数の変化
- 開発プロセスとコードメトリクスの関係性を解明
 - コード自体の変化と開発者の行動を対応付け、具体的な改善策を提示

3. 関連研究

- レビューコメントを収集し、保守性の低下リスクを分析^[2]
- 静的解析ツールを用いてコードの改善策を提示^[3]
- コードメトリクスを用いたバグ予測により、コード品質を改善^[4]

[2] X.Han et al., "Understanding Code Smell Detection via Code Review: A Study of the OpenStack Community", 2021

[3] S.Romano et al., "Do Static Analysis Tools Affect Software Quality when Using Test-driven Development?", 2022

[4] R.Ferenc et al., "An automatically created novel bug dataset and its validation in bug prediction", 2020

4. 分析手順

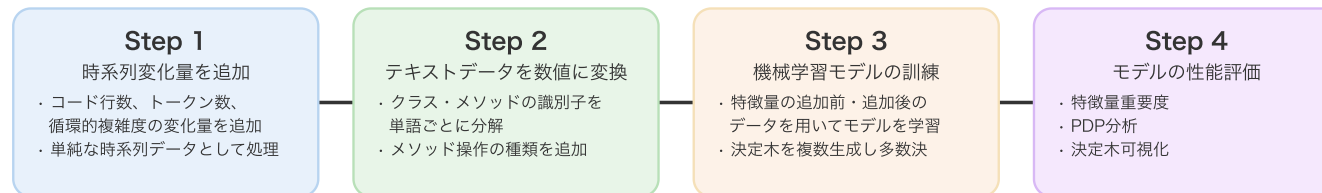


図2 データ分析の流れ

1. 時系列変化量を追加
2. テキストデータを数値に変換
3. 特徴量の追加前・追加後のデータを用いて機械学習モデルを訓練
4. 各プロジェクトでモデルの性能を評価

4.1 時系列変化量の追加

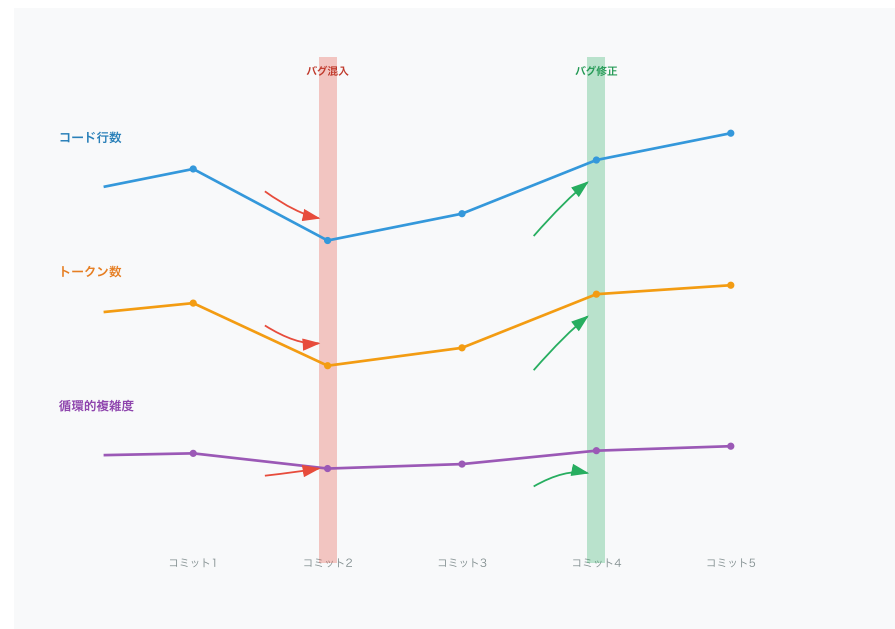


図3 ソフトウェア構造の変化

- 単純な時系列データとして、コード行数・トークン数・循環的複雑度の変化量をデータセットに追加

4.2 テキストデータの数値変換

- クラスやメソッドの識別子を単語ごとに分解
 - 単語の出現率などの類似性に基づいて学習できるようにする
- メソッドの変更履歴（例: 新規追加、変更、削除）を表す変数を導入
 - 変化量が欠損した理由を説明できるようにする

4.3 機械学習モデルの訓練

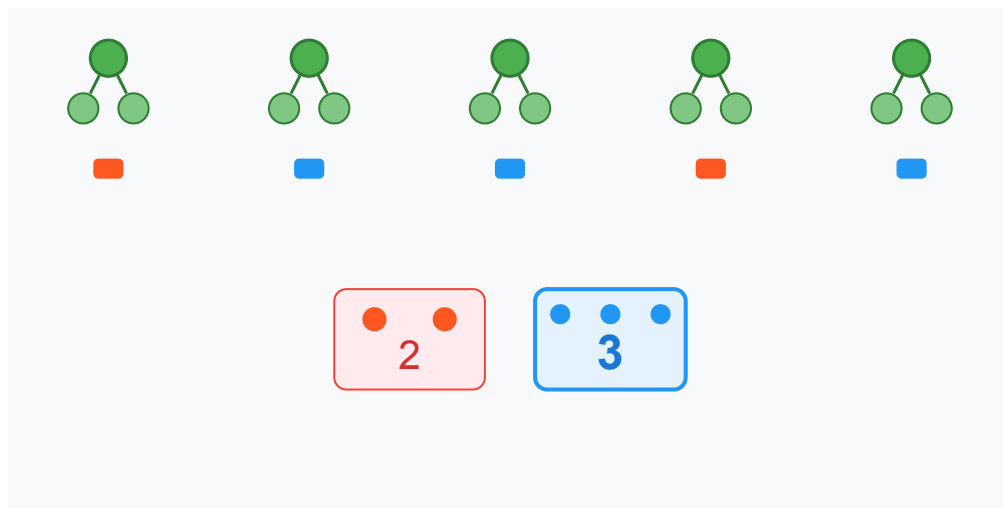


図4 複数の決定木を用いた投票

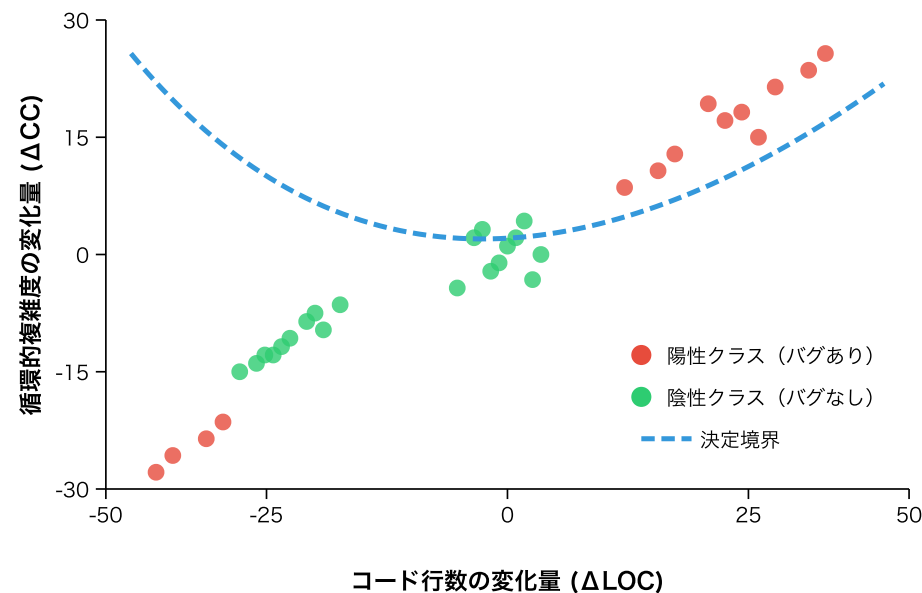


図5 決定境界による2値分類

- 決定木を複数生成し、多数決で境界線を決定

4.4 プロジェクトごとの性能評価

表1 選定したプロジェクト

プロジェクト	コード行数 (概算)	コミット数	バグレポートの数
ANTLR v4	68,000	6,526	179
Broadleaf Commerce	322,000	14,920	703
Eclipse plugin for Ceylon	181,000	7,984	923
Elasticsearch	995,000	28,815	4,494
Hazelcast	949,000	24,380	3,882
Oryx	34,000	1,054	67

4.4 プロジェクトごとの性能評価

1. 特徴量重要度 (Feature Importance) を算出
2. ヒストグラムで特徴量分布を確認
3. Partial Dependence Plot (PDP) を用いて分類傾向を把握
4. 決定木で分類の流れを可視化し、判断の基準と信頼性を確認

5.1 特徴量重要度

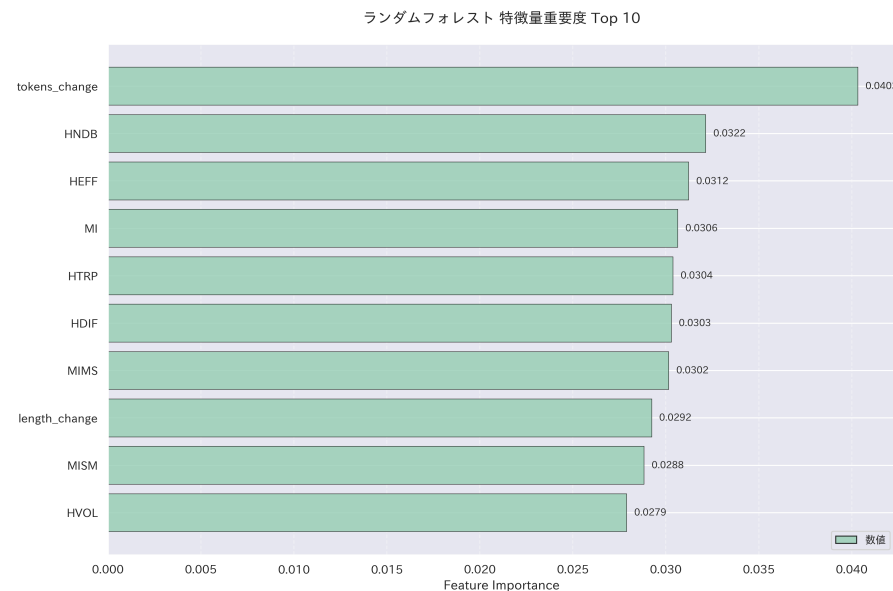


図6 Hazelcastの特徴量重要度

- トークン数・コード行数の変化量、Halsteadメトリクス、Maintainability Indexの重要度が高い

5.2 特徴量分布

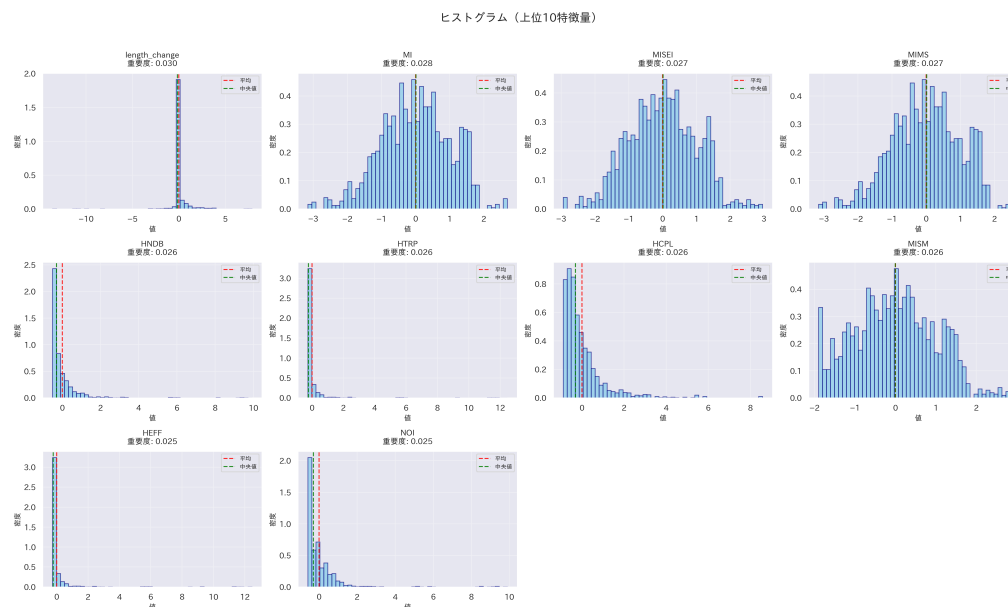


図7 Eclipse plugin for Ceylonの
特徴量分布

- 変化量やHalstead系は分散が小さく、Maintainability Indexは分散が大きい

5.3 PDP分析

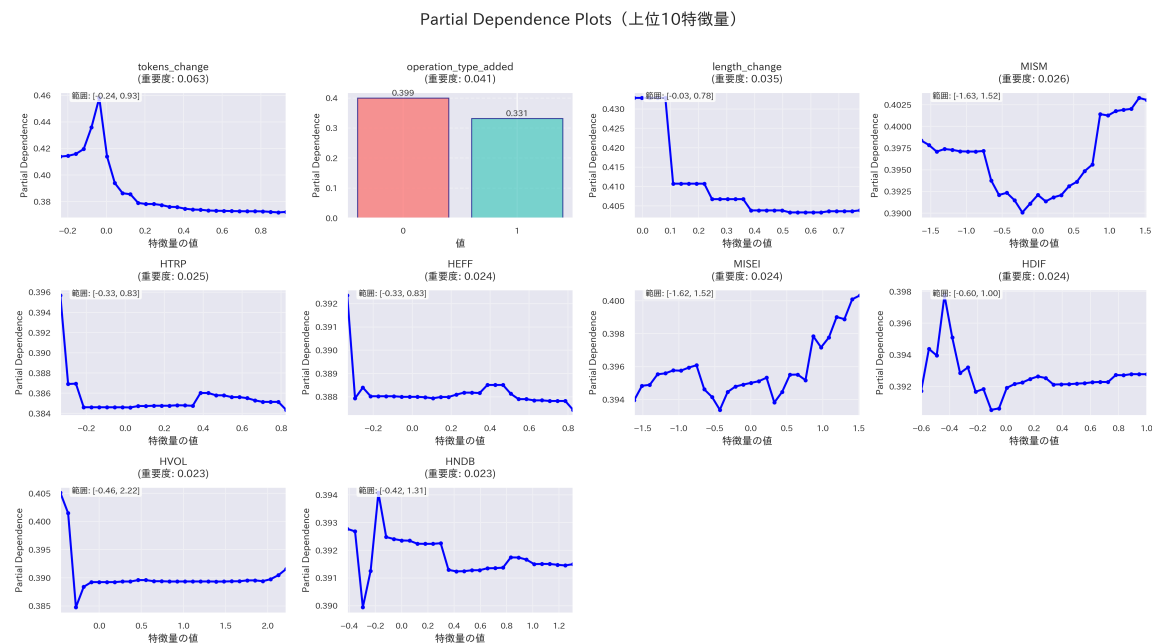


図8 ElasticsearchのPDP

- ほとんどの特徴量において、陽性クラスの予測確率が0.5未満

5.4 決定木分析

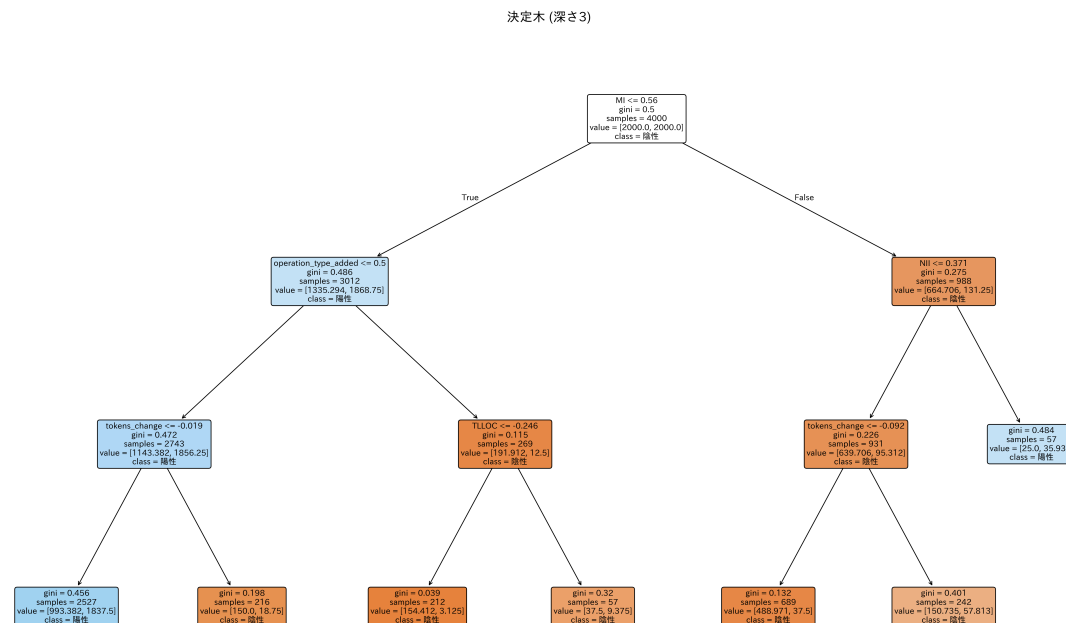


図9 Elasticsearchの決定木

- 陰性クラスの子ノードのジニ不純度が比較的低い

5.5 評価指標の測定

表2 評価指標の値の変化

プロジェクト	F1スコア (変更前)	F1スコア (変更後)
Eclipse plugin for Ceylon	0.39	0.49
Elasticsearch	0.62	0.72
Hazelcast	0.67	0.71
ANTLR v4	0.45	0.48
Oryx	0.33	0.39

モデルの予測性能が改善される傾向があることを確認

6. 考察

陰性クラス予測の改善

リファクタリングによるメトリクスの減少傾向を確認できた

消去法的な分類がF1スコアの改善に寄与

さらなる精度向上に向けて

PDPや決定木を見ると、陽性クラスの予測確率が低い

バグの混入理由が多様であることが影響している可能性

7. 課題と展望

開発プロセスの理解

レビュー記録や自動テストの内容からバグが生じる状況を説明

陽性クラスの詳細な分類による因果関係の解明

開発プロセスやコードメトリクスの変化がバグにどのように影響するかを明らかにする

まとめ

半数のプロジェクトで有意差があり、F1スコアが最大0.1向上

コードメトリクスの変化量を組み合わせることで、より効果的なバグ分類ができるようになった

今後は陽性クラスの分類精度を高め、品質改善に役立つ特徴を探す